

PRD: Distributed Tatkal Booking Engine

Project Vision

Build a backend-only distributed system that simulates the Tatkal railway booking rush that occurs at 11:00 AM when thousands of users attempt to book a limited number of seats simultaneously.

The project is not intended to be a production IRCTC clone.

The goal is to learn and demonstrate:

- Backend Engineering
- System Design
- Concurrency Handling
- Distributed Systems
- Docker
- Kubernetes
- Horizontal Scaling
- Load Testing
- Architecture Evolution

The project should intentionally start as a Monolith and later evolve into a Microservice Architecture to demonstrate the limitations of monoliths under load and the benefits of service isolation and autoscaling.

Primary Business Scenario

Train:

Mumbai → Delhi

Available Tatkal Seats:

500

Simulated Users:

10,000

At exactly:

11:00:00 AM

Thousands of users attempt to:

1. Login
2. Check Availability
3. Book a Seat

The system must survive the traffic spike while maintaining correctness.

Project Philosophy

This is NOT:

- Full IRCTC Clone
- Frontend Project
- UI/UX Project
- Kafka Project
- Event Sourcing Project
- Production Railway Reservation System

This IS:

- Backend Architecture Project
- Learning Project
- Distributed Systems Project
- Kubernetes Autoscaling Project
- Concurrency Control Project

Keep architecture intentionally simple and focused.

Core Engineering Problems To Solve

Problem 1: Double Booking

Scenario:

Only 1 seat remains.

Two users click Book simultaneously.

Bad Result:

Seat sold twice.

Required Result:

Only one booking succeeds.

System must guarantee:

- No overselling
 - No duplicate seat allocation
-

Problem 2: Auth CPU Spike

Scenario:

Thousands of login requests occur simultaneously.

bcrypt hashing consumes CPU.

Bad Result:

Booking APIs become slow.

Required Result:

Authentication workload should not affect booking operations.

Problem 3: Traffic Spike

Scenario:

Normal traffic = 100 requests/sec

Tatkal traffic = thousands of requests/sec

Required Result:

System automatically scales.

Problem 4: Retry Storm

Scenario:

User clicks Book multiple times.

Network delays cause repeated requests.

Required Result:

Only one booking should be created.

Introduce Idempotency concepts.

Problem 5: Service Isolation

Scenario:

Auth Service overloaded.

Required Result:

Seat and Booking services continue operating normally.

Functional Requirements

User Registration

Users can register.

Fields:

- Name
- Email
- Password

Passwords must be hashed.

User Login

Users can login.

JWT authentication.

Availability Check

Users can check remaining seats.

Returns:

- Train Id
 - Total Seats
 - Available Seats
-

Book Seat

Authenticated users can book a seat.

System must:

- Verify availability
 - Prevent overselling
 - Create booking
-

View Booking

User can retrieve booking details.

Non Functional Requirements

Consistency

No double booking.

No overselling.

Availability

System should remain operational during spikes.

Scalability

Support autoscaling through Kubernetes HPA.

Low Latency

Seat availability should remain fast even during login spikes.

Phase 1: Monolith Architecture

Build first version as a Monolith.

Architecture:

|
MongoDB

Purpose:

Demonstrate limitations.

Expected Failures:

- CPU bottlenecks
 - Shared resource contention
 - Single point of failure
-

Monolith Documentation Requirements

Generate:

1. Monolith Architecture Diagram
2. Monolith Component Diagram
3. Monolith ER Diagram
4. Monolith API Documentation
5. Monolith Sequence Diagrams

Sequence Diagrams Required:

- Login Flow
- Seat Availability Flow

- Booking Flow
 - Concurrent Booking Flow
-

Monolith Load Testing

Use k6.

Test Cases:

1. Login Flood
2. Availability Flood
3. Booking Flood

Collect:

- Throughput
- CPU Usage
- Memory Usage
- Latency

Document failures.

Create:

Failure Analysis Report

Questions:

- Why did latency increase?
 - What service became bottleneck?
 - What architectural weakness was exposed?
-

Phase 2: Microservice Migration

Refactor monolith into microservices.

Architecture:

API Gateway

|

MongoDB

Redis

Service Responsibilities

Auth Service

Responsibilities:

- Register
- Login
- JWT

Purpose:

Generate CPU intensive workload.

Seat Service

Responsibilities:

- Availability
- Seat inventory

Purpose:

Fast reads.

Equivalent to stock-service in flash sale architecture.

Booking Service

Responsibilities:

- Booking creation
- Seat reservation
- Concurrency handling

Purpose:

Critical transactional service.

API Gateway

Responsibilities:

- Routing
- Request forwarding

Purpose:

Single entry point.

Redis Requirements

Use Redis for:

1. Seat Inventory
2. Atomic Seat Decrement
3. Idempotency Keys

Demonstrate atomic operations.

Goal:

Prevent overselling.

MongoDB Requirements

Store:

Users

Bookings

Seats

Train Information

Microservice Documentation Requirements

Generate:

1. Microservice Architecture Diagram
 2. Service Communication Diagram
 3. Updated ER Diagram
 4. API Contracts
 5. Deployment Diagram
 6. Container Diagram
-

Docker Phase

Containerize:

- API Gateway
- Auth Service
- Booking Service
- Seat Service
- MongoDB
- Redis

Create:

docker-compose.yml

Generate container communication diagram.

Kubernetes Phase

Deploy services to local Kubernetes cluster.

Use:

- Deployments
- Services
- Ingress
- HPA

Generate Kubernetes Architecture Diagram.

Autoscaling Requirements

Configure HPA:

Target CPU:

50%

Minimum Pods:

1

Maximum Pods:

5

Demonstrate automatic scaling under load.

Load Testing Phase

Use k6.

Test:

500 concurrent users

Train Capacity:

50 seats

Expected:

50 successful bookings

450 failures

No overselling.

Metrics To Capture

For Monolith:

- CPU
- Memory
- Throughput
- Latency

For Microservices:

- CPU per Service
- Throughput per Service
- Scaling Events
- Pod Count

Deliverables

Repository Structure:

distributed-tatkal-booking-engine

docs/
monolith/
microservices/
k8s/
scripts/
diagrams/

Required Documentation:

- Problem Statement
- Requirements
- Architecture Decisions
- Monolith Architecture
- Problem in monolith /report ex: traffic spike , double payment
- Microservice Architecture
- ER Diagrams
- Sequence Diagrams
- Failure Analysis
- Docker Design
- Kubernetes Design
- Load Test Results
- Benchmark Report

Learning Mode Instructions For AI

Before generating code:

1. Explain architecture.
2. Explain service responsibilities.
3. Explain database design.
4. Explain communication flow.
5. Explain sequence diagrams.
6. Explain tradeoffs.

Only after explanation:

Generate code.

For every generated file:

Explain:

- Why it exists
- Who calls it
- What problem it solves
- Which service depends on it

The goal is understanding architecture first and implementation second.

Success Criteria

The project is successful if:

- Monolith version works
- Monolith limitations are documented
- Microservice version works
- Redis prevents overselling
- Kubernetes autoscaling is demonstrated
- Load tests prove service isolation
- Architecture evolution is clearly documented

The repository should tell the story:

Problem → Monolith → Failure → Microservices → Kubernetes → Autoscaling → Results